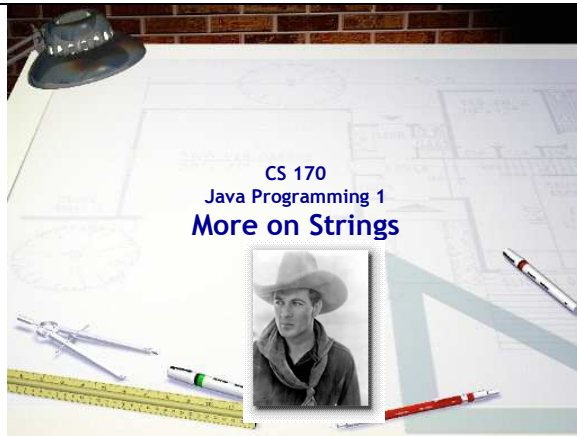




Slide 1
CS 170
Java Programming 1
More on Strings
Duration: 00:00:47
Advance mode: Auto

Notes:

Even Gary Cooper, a man of notoriously few words, was not reduced to speaking in single characters. Most communication requires groups of characters. In human speech we call these phrases and sentences. In Java, we store phrases and sentences in Strings.

In Chapter 1 of your text, you met the String class and worked with literals, output, and concatenation. In this lesson, (and in Chapter 4 of your text), we'll take a look at the methods available to String objects. Let's start with a little String refresher.

- What are **Strings** in Java?
 - **Immutable** sequences of 0...n characters
- **Strings** are like primitives in several ways
 - Initialize using a literal rather than using **new**
- **Strings really** are objects, though, not primitives
 - Strings are references not values

Slide 2
Meet the Strings
Duration: 00:02:46
Advance mode: Auto

Notes:

In Java, the definition of a String is: an immutable sequence of 0..n characters.

There are two things to notice about this definition that makes Java String objects different than characters, and different than strings in other programming languages.

- First, Java strings are immutable or constant. Just as the character literal 'A' cannot suddenly morph into 'B', so a String object, once created, can never be changed. It will always be composed of exactly the same sequence of characters.

This is much different than C or Pascal, where you can change the individual characters that a string contains. Java has the StringBuffer class, which works more like strings in other languages.

- Second, Java strings can hold 0..n characters, while a char variable must always hold exactly one character.

Let's take a closer look at the nature of the String class.

Because Java has a strong syntactical resemblance to the C programming language, programmers often expect Java String objects to act like "null-terminated arrays of char", which is what strings are in C. Java's String objects don't act like that at all!

In fact, Java Strings act a little bit like objects, (but with some unusual features), and a little bit like primitive types. String objects, for instance, are the only object type that allows you to initialize them using literals, without using the new operator.

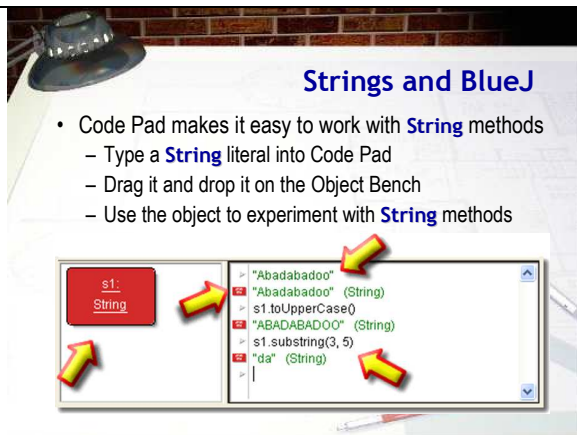
Furthermore, unlike the other object types, String objects can be manipulated with several different operators.

In reality, though, Strings are not primitive or fundamental

types. Unlike primitive types, String variables hold references to Strings, not the actual character values. That means that two String variables can refer to the same, actual String object like that shown in the illustration here.

As you can imagine, if the String variable named first were allowed to change the contents of the String object, "Hello Dolly", it would have a serious impact on the String variable named second. That's the reason that String objects are immutable in Java.

This is not to say that you can't manipulate a String. Unlike the numeric and character primitives, the String has a rich variety of methods that can be used to control them; they are not restricted to operator manipulation. You can find a list of the basic methods in Appendix C, on page 1080 in your textbook. For the complete listing of all the methods, use the online JavaDocs (as always).



The screenshot shows the BlueJ IDE. On the left, the 'Object Bench' contains a red icon labeled 's1: String'. On the right, the 'Code Pad' contains the following code:

```
> "Abadabadoo" (String)
> s1.toUpperCase()
> "ABADABADOO" (String)
> s1.substring(3, 5)
> "da" (String)
```

Yellow arrows indicate the workflow: one arrow points from the 's1: String' icon to the first line of code, and another points from the 'da' result back to the icon. A third arrow points from the 'da' result to the 'Object Bench'.

Slide 3

Strings and BlueJ

Duration: 00:01:18

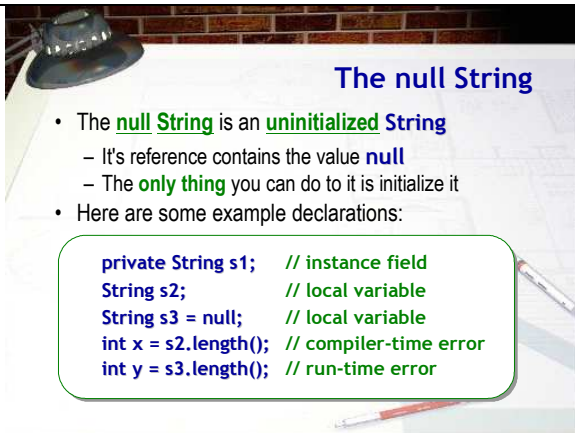
Advance mode: Auto

Notes:

Instead of looking up all of the methods in the String class, you can use Code Pad and BlueJ's Object Bench to interactively look up and try any methods you're interested in. To do that:

- Type a String literal into the Code Pad. You don't need a variable name or a semicolon. Note that when you do this, BlueJ evaluates the literal as a simple expression and shows you its value and type, just like it does with integers and real numbers. With object types, though, you'll notice a small red miniature object icon in the Code Pad margin immediately to the left of the evaluation.
- Drag the object icon and drop it on the Object Bench. BlueJ asks you to provide a named for the new object variable you've placed on the bench. In my picture here, I've named my String s1.

Now, you can experiment with the new reference, just as if you'd created an object variable in Code Pad. Notice that I've used the variable to try out the substring() method. Even better, though, is the fact that you can now right-click the icon on the Object Bench and use the menus to call it's methods interactively.



The null String

- The **null String** is an **uninitialized String**
 - It's reference contains the value **null**
 - The **only thing** you can do to it is initialize it
- Here are some example declarations:

```
private String s1; // instance field
String s2;        // local variable
String s3 = null; // local variable
int x = s2.length(); // compiler-time error
int y = s3.length(); // run-time error
```

Slide 4

The null String

Duration: 00:01:28

Advance mode: Auto

Notes:

You'll recall that to initialize a String variable using a literal, just form your String using regular characters, and enclose the whole thing in double quotes. If you don't initialize a String variable when you create it, you have a String variable that points to "nothing". This is called the null String. You cannot send any messages to a null String or apply any operations to a null String, except for the assignment operator.

Here are three definitions that illustrate the null String:

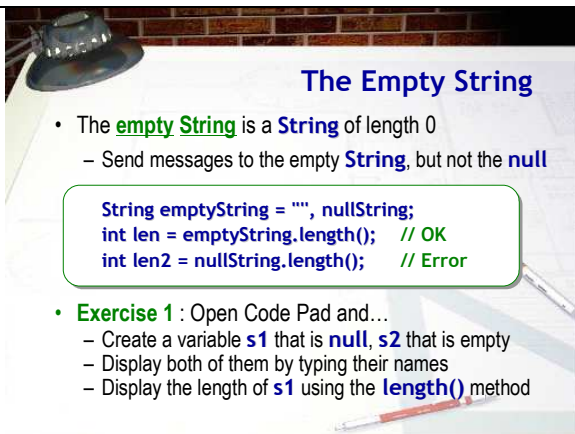
```
Private String s1; // Instance variable not
                  // initialized; value is null
```

```
String s2; // Local variable not initialized
```

```
String s3 = null; // Local variable not initialized
```

In the first case, the String s1 is an instance variable not assigned a value, so it is given the value null by default. In the second example, the local variable s2 is not assigned a value at all, so it is also null. Finally, the special object constant value null is assigned to the local String variable s3.

Note that since the String s2 is a local variable, Java won't compile your code if you attempt to use the variable, such as this line which attempts to find its length. With both s1 and s3, though, the Java compiler won't prevent your code from compiling, but a runtime error will occur when you use either variable (except for assignment).



The Empty String

- The **empty String** is a **String** of length 0
 - Send messages to the empty **String**, but not the **null**

```
String emptyString = "", nullString;
int len = emptyString.length(); // OK
int len2 = nullString.length(); // Error
```

- Exercise 1**: Open Code Pad and...
 - Create a variable **s1** that is **null**, **s2** that is empty
 - Display both of them by typing their names
 - Display the length of **s1** using the **length()** method

Slide 5

The Empty String

Duration: 00:01:12

Advance mode: Auto

Notes:

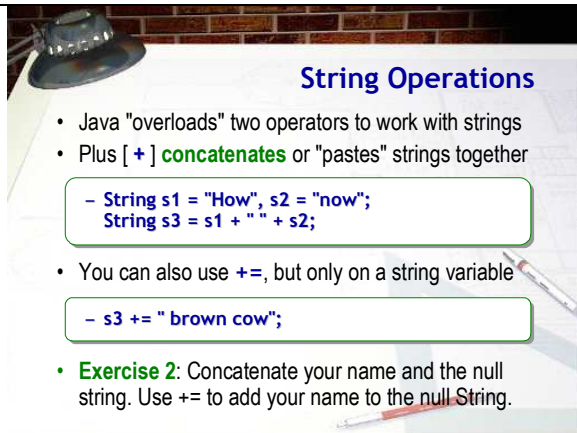
A null String does not refer to anything. The empty String, in contrast, refers to a valid String object, one that contains zero characters. Unlike the null String, the empty String responds to messages and may be used in String expressions.

To create an empty String you use a pair of adjacent double-quotes with no intervening spaces like this: String emptyString = ""; Make sure you don't put a space in between the quotes.

OK, let's try both in Code Pad. Create a new section in this week's exercise document, open Code Pad, and for Exercise 1:

- Create two variables. s1 is a null String and s2 is an empty String
- Display both of their values by simply typing their names (one per line) and pressing Enter
- See if I'm right about the methods. Use the length() method to try to display the length of s1.

When you're done, shoot a screen-shot of your Code Pad session.



String Operations

- Java "overloads" two operators to work with strings
- Plus [+] **concatenates** or "pastes" strings together


```
String s1 = "How", s2 = "now";
String s3 = s1 + " " + s2;
```
- You can also use +=, but only on a string variable


```
s3 += " brown cow";
```
- Exercise 2:** Concatenate your name and the null string. Use += to add your name to the null String.

Slide 6 🎧
String Operations
 Duration: 00:01:35
 Advance mode: Auto

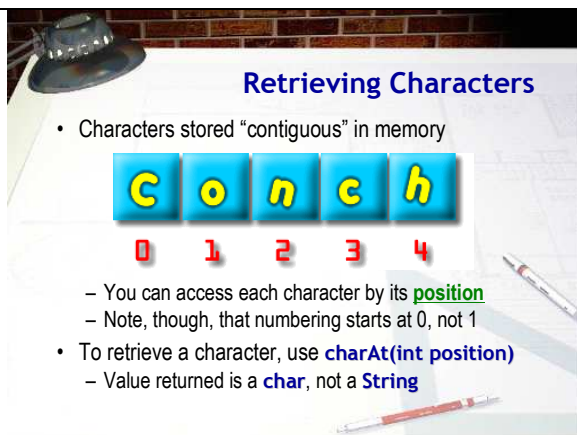
Notes:

Although Strings are really reference types, like other objects, there are also have several operators that act upon String operands.

When used with Strings, the "plus sign" (+) acts to "paste" two Strings together to form a new String, a process called concatenation. Notice with s3, concatenation works with both String variables (s1 and s2) and String literals (the empty space concatenated between the two words.)

In addition, the shorthand operator += also performs concatenation when its left-hand operand is a String variable. Here I've created a new String by concatenating "(space) brown cow" to the end of the String s3 and then storing the results back in the String variable s3. Note that when you do this, the original contents that s3 referred to are not changed! An entirely new String is constructed and the variable s3 now points to the new String.

For Exercise 2, let's try out some String concatenation. What happens if you concatenate your name and a null String? What happens if you use += to add your name to a null String (that is, if the null String is the variable that appears on the left of the short-hand assignment operator)? Try both of these and shoot a screen-shot.



Retrieving Characters

- Characters stored "contiguous" in memory

C	o	n	c	h
0	1	2	3	4
- You can access each character by its **position**
 - Note, though, that numbering starts at 0, not 1
- To retrieve a character, use **charAt(int position)**
 - Value returned is a **char**, not a **String**

Slide 7 🎧
Retrieving Characters
 Duration: 00:01:53
 Advance mode: Auto

Notes:

When you first met the String class, you saw that Strings had some of the characteristics of primitives and some characteristics of objects. One of the ways in which Strings are like objects is that the String class has a rich set of methods you can use to manipulate String objects.

You've already seen the String length() method. Let's look at a few of the others.

Suppose you created a String variable, s1, and initialized it with the word "conch". If you were to "peer inside" the String object referred to by the variable s1, you'd see that the individual characters were stored right next to each other in memory. Because of this, Java lets you access the individual characters by position, using an index value.

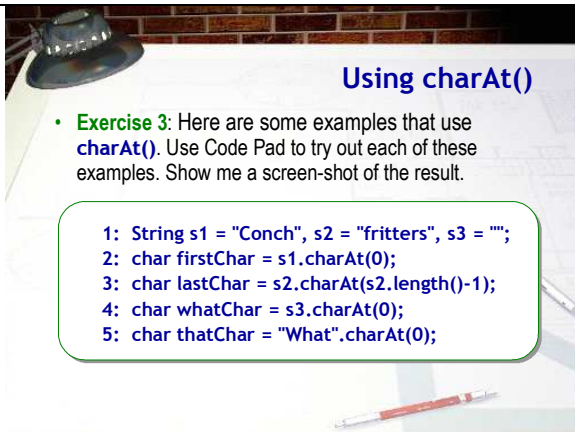
Most other languages use a similar scheme, but some languages start their counting at 1, (VB and Pascal), and others start their counting at 0, (Java and C++). As you can see from the picture shown here, the first character in a Java String, like the capital "C" in "Conch", is associated with the index position 0.

To retrieve the individual characters that make up a String, use the String method: charAt(int position). Pass your variable the index position of the character you want, and the charAt() method return the character itself.

There are two things you should be careful about. First, the value returned from charAt() is a char, not a 1-character

String. If you want to use the returned value with other strings, you'll have to use concatenation to convert it back into a String object.

Second, the last character in any Java String will always be a position `s1.length() - 1`.



Using charAt()

- **Exercise 3:** Here are some examples that use `charAt()`. Use Code Pad to try out each of these examples. Show me a screen-shot of the result.

```
1: String s1 = "Conch", s2 = "fritters", s3 = "";
2: char firstChar = s1.charAt(0);
3: char lastChar = s2.charAt(s2.length()-1);
4: char whatChar = s3.charAt(0);
5: char thatChar = "What".charAt(0);
```

Slide 8

Using charAt()

Duration: 00:01:33

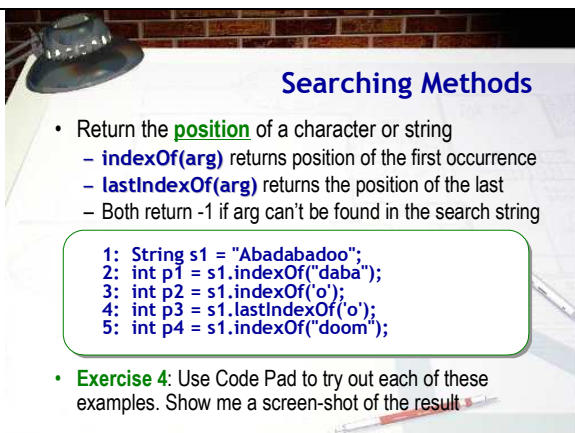
Advance mode: Auto

Notes:

Let's try some hands-on exercises with Code Pad for Exercise 3. Below, there are a couple of examples using the `charAt()` method. Pause the lecture, type each of them in, and show me a screen-shot of the result. Before you type each one in, though, see if you can figure out what the result should be. Then display the result from each variable.

Here's some analysis you can use to check your work.

- Line 2 stores the character 'C' in the variable `firstChar`, because the first character uses the index value 0.
- Line 3 uses `charAt()` to retrieve the last character in "fritters", and then stores it in the char variable `lastChar`. Note that `length()-1` is used as the index value passed to `charAt()`.
- Line 4 causes a runtime error since the empty String, `s3`, does not have a character at position 0. Note that this is different than an error caused by the null String. You can send messages to the empty String, but you can't ask it to give you a character that it doesn't possess.
- Line 5 stores the character 'W' in the variable `thatChar`. Note that you can call methods using String literals as the receiver instead of String variables.



Searching Methods

- Return the **position** of a character or string
 - `indexOf(arg)` returns position of the first occurrence
 - `lastIndexOf(arg)` returns the position of the last
 - Both return -1 if arg can't be found in the search string

```
1: String s1 = "Abadabadoo";
2: int p1 = s1.indexOf("daba");
3: int p2 = s1.indexOf('o');
4: int p3 = s1.lastIndexOf('o');
5: int p4 = s1.indexOf("doom");
```

- **Exercise 4:** Use Code Pad to try out each of these examples. Show me a screen-shot of the result

Slide 9

Searching Methods

Notes:

The `charAt()` method is useful when you know the position of a particular value. Often, though, with user-supplied data, you'll need to search through the string to find the value you want. The String class has several methods that will let you search for specific values, or, the positions of specific values.

There are several versions of the `indexOf()` and `lastIndexOf()` methods, which search the String on the left, (the one calling the method), for the value on the right, (the parameter passed to the method). The parameter can be either a character or a String.

The `indexOf()` methods start searching at the beginning of the String, and the `lastIndexOf()` methods start searching at the end of the String. Both methods return the index position,

Duration: 00:03:36
Advance mode: Auto

measured from the beginning of the string, where the searched-for value was found. Both methods also return -1 if the character or String searched for is not found.

Here are some examples. For Exercise 4, use Code Pad to try out each line and print the results of the variables p1, p2, p3 and p4. As you did before, try to figure out what will be printed before you display the results.

Here's some analysis of this exercise.

- Line 1 stores the address of the literal "Abadabadoo" in the variable s1.
- Line 2 uses indexOf() to retrieve the index position of the word "daba" contained inside s1. Since the first character is at position 0, you can see that the phrase starts at position 3 which is stored in the int variable p1.
- *Line 3 also uses the indexOf() method, only this time the method searches for a char parameter, not a String. "Abadabadoo" contains the character 'o' twice, in positions 8 and 9. The indexOf() method returns the position of the first match it finds.

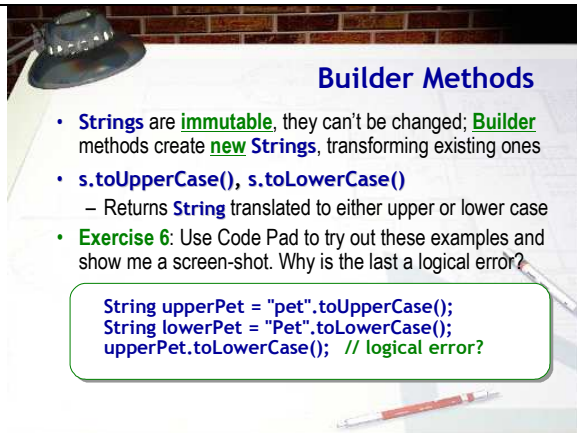
This is actually a **different version** of the indexOf() method than that used in Line 2, since each method has to specify what kind of parameters it requires. That means a method cannot be written so that a user can provide either a String or a char. To get the same effect, programmers can write different versions of the same method, using the same name. Different versions of a method, using the same name but requiring different parameters, are known as overloaded methods.

- Line 4 also searches for the character 'o', but this time a version of the lastIndexOf() method is used. As its name suggests, this method starts searching from the end of the String until it finds a match. Since it finds the character 'o' immediately at the end of the string, it returns its position, 9, which is stored in the variable p3.
- Finally, Line 5 searches the Abadabadoo for the word "doom". Since doom occurs nowhere in the string, the variable p4 is given the value -1.

Two more searching methods that will prove useful once you learn about selection, (since they return boolean values), are:

endsWith(suffix), startsWith(prefix)

Both of these methods return true if the String on the left, (the String calling the method), ends with or starts with the String on the right, (the String passed as an argument).



Builder Methods

- **Strings** are **immutable**, they can't be changed; **Builder** methods create **new Strings**, transforming existing ones
- **s.toUpperCase(), s.toLowerCase()**
 - Returns **String** translated to either upper or lower case
- **Exercise 6:** Use Code Pad to try out these examples and show me a screen-shot. Why is the last a logical error?

```
String upperPet = "pet".toUpperCase();
String lowerPet = "Pet".toLowerCase();
upperPet.toLowerCase(); // logical error?
```

Slide 10 
Builder Methods
 Duration: 00:01:24
 Advance mode: Auto

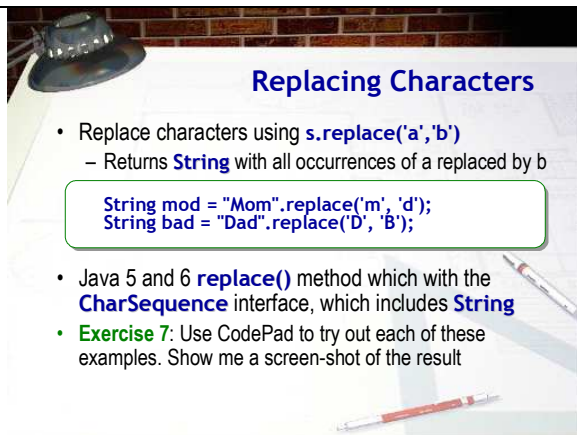
Notes:

Because Strings are immutable, the Java String class has no mutator methods. In place of these, however, there are a number of "builder" methods that produce a new String by transforming an existing String. When you use any of these transforming builder methods, you have to remember to save the results using a String variable.

Two of the methods you'll use often are toUpperCase() and toLowerCase(). These return the String on the left, (the one calling the method or the implicit parameter), translated into either upper or lower case.

For Exercise 6, use Code Pad to create try out the lines below and display the contents of each variable after you execute each line. Shoot me a screen-shot of the result. Before you do that, though, see if you can figure out why the last line has the comment about a logical error.

Done? The first two lines in this example call a transforming method and store the results in a new variable. The last line shows a common logic error; the programmer thought the code was converting upperPet to lowercase, but instead it creates a new String object and then simply abandons the new object altogether. The variable upperPet is not modified.



Replacing Characters

- Replace characters using **s.replace('a','b')**
 - Returns **String** with all occurrences of a replaced by b

```
String mod = "Mom".replace('m', 'd');
String bad = "Dad".replace('D', 'B');
```

- Java 5 and 6 **replace()** method which with the **CharSequence** interface, which includes **String**
- **Exercise 7:** Use CodePad to try out each of these examples. Show me a screen-shot of the result

Slide 11 
Replacing Characters
 Duration: 00:01:19
 Advance mode: Auto

Notes:

Another builder method returns a String with all occurrences of character a replaced by character b, not just the first occurrence. The replace method works like this. Note that in the first example, the variable mod should contain the word "Mod" because we replaced the lowercase 'm' with the lowercase 'd'. In the second, the variable bad should contain the word "Bad".

Note that there is no replace() method that takes String arguments, like there is for the searching methods. Java 1.4 added a ReplaceAll() method, which works with regular expressions, and Java 5 and 6 both contain a version of replace() that works on classes implementing the CharSequence interface, which the String class does. The bottom line? You can use replace with String arguments in Java 5 and 6 but if you want your code to work in 1.4, use replaceAll().

For Exercise 7, type both of these examples into Code Pad and then display the contents of the variables mod and bad just to check that they work as you'd expect. Print a screen-shot of the contents of the variables.

Substrings

- Substring methods **extract** a portion of a string, given a starting index and an (optional) ending index
 - `s.substring(start)`, `s.substring(start,end)`
- Version 2 returns substring starting (and **including**) position **start**, up to, and **excluding**, position **end**

```
1: String s1 = "Hello, World!";  
2: String s2 = s1.substring(7);      // World!  
3: String s3 = s1.substring(7, 12); // World
```



Slide 12

Substrings

Duration: 00:01:53

Advance mode: Auto

Notes:

The last builder method we'll look at is substring, which returns a new String using a portion of the String on the left. There are two versions of substring: both take a starting index value. The returned String starts with the element passed as start, with the first element being 0.

The second version returns the portion of the String starting (and including) position start, up to, and excluding, position end. Here's an example.

- Line 1 creates the String variable s1 and assigns it to the literal "Hello World!".
- Line 2 uses the substring() method to extract the portion of s1 starting with the character at position 7. The returned String includes everything from position 7 onward to the end of the String.
- Line 3 uses the overloaded version of substring() to extract the portion of s1 starting at position 7, and going up to, but not including, the character at position 12.

The fact that you need to supply two index values is a little awkward. One nice thing, though, is that you can easily use the two index values to compute the length of the substring, by subtracting the starting index from the ending index.

Also, if you want to extract a substring of a particular size, just do it by passing an expression using the first index plus the desired length as the second parameter. We could rewrite Line 3, as `String s3 = s1.substring(7, 7 + 5)` instead.

Express Yourself

- Exercise 8:** In Code Pad create three **String** variables, **name1**, **name2**, and **name3**, initialized to "Last,First", "Last, First", and "Last , First" using your own name.
 - You can use the **String trim()** method to remove extra spaces before and at the end of a string. You know that name contains a comma, but you aren't sure if there are spaces before or after the comma or not.
 - Use the **indexOf()**, **substring()** and **trim()** functions to extract the first and last name into two new variables: **first** and **last** using **name1**, **name2** and **name3**. Print the values of **first** and **last**, surrounded by double quotes after each extraction.
 - Show me a screen-shot of the result

Slide 13

Express Yourself

Duration: 00:01:36

Advance mode: Auto

Notes:

OK, let's put all of this together. In Code Pad, create three String variables, name1, name2, and name3. Initialize them using "Last,First", "Last, First", and "Last , First", replace Last and First with your name.

If a String contains spaces either at the beginning or end you can remove them using the trim() method. This is a builder method, so you have to remember to assign the results back into an appropriate variable. You know that the variables name1, name2 and name3 all contain a comma separating the two names, but you don't know if there are any spaces either before or after the comma.

Use the indexOf(), substring() and trim() functions to extract the first and last name (with no extra spaces) into two new variables first and last. Use exactly the same code to extract the names on the variables name1, name2 and name3. Don't customize your code for the spaces that the String contains. After each extraction, print the values of first and last, surrounded by double-quotes so I can see that you've removed the extra spaces.

Show me a screen-shot of the result.

